

Deliverable D3: Project Technical Report

1. Executive Summary

This report presents the implementation of a comprehensive Mini Compiler for a subset of the Pascal programming language. The compiler incorporates a double-buffered lexical analyzer, three independent parsing engines (Recursive Descent, table-driven Predictive LL(1), and shift-reduce SLR(1) LR), a hash-based scope-nested symbol table, and an error accumulator with panic-mode recovery. The entire project is written in standard C++17.

2. Description of Compiler Modules

2.1 Lexical Analyzer (`lexer.cpp`)

The Lexer reads raw source files using a **double-buffering** technique. It maintains two alternating buffers of 4096 bytes:

- A `cursor` pointer scans the active buffer character-by-character.
- When the cursor reaches the boundary of the active buffer, the next block of the file is loaded into the other buffer, and the active buffer index switches.
- This ensures $O(1)$ memory overhead regardless of the source file size.
- Comment structures (`{ ... }` and `(* ... *)`) are skipped, and whitespace is ignored.
- Coordinates (line and column numbers) are tracked for every matched token.

2.2 Grammar Analysis Engine (`grammar.cpp`)

Instead of hardcoding static parsing matrices, the Grammar module dynamically computes parser parameters:

- **FIRST and FOLLOW Sets:** Calculated iteratively by applying grammar derivation equations until a fixed point is reached.
- **LL(1) Table Construction:** Maps `[NonTerminal][Terminal]` to production indices.
- **SLR(1) Table Construction:**
 - Creates the LR(0) collection of item sets using standard closure and GOTO operations.
 - Computes states dynamically.
 - Resolves conflicts: Shift-Reduce conflicts (e.g. dangling else) are resolved by prioritizing **Shift** actions. Reduce-Reduce conflicts are resolved by choosing the first declared production index.

2.3 Recursive Descent Parser & Semantic Analyzer (`parser_rd.cpp`)

- Implements a recursive method for each non-terminal.
- Validates syntax by matching tokens.
- Simultaneously performs **Semantic Analysis**:
 - Validates scopes and detects duplicate variables.
 - Verifies that referenced variables have been declared.
 - Checks function parameters (count and type validation).
 - Prevents assignments to constants.
- Generates an Abstract Syntax Tree (AST) representing the program.

2.4 LL(1) Non-Recursive Predictive Parser (`parser_ll.cpp`)

- Runs a stack-based parsing loop using the program's explicit LL(1) table.
- Employs **panic-mode recovery**: if a non-terminal A cannot derive the lookahead token a , the parser skips input tokens until it finds a token a in $\text{FOLLOW}[A]$. It then pops A from the stack and resumes parsing.

2.5 SLR(1) LR Shift-Reduce Parser (`parser_lr.cpp`)

- Executes the shift-reduce algorithm using a state stack and a symbol stack.
- Actions are determined by looking up current state and token in the SLR(1) action table.
- Writes a detailed trace of stack states and lookahead symbols to a trace log file.
- Panic-mode recovery: search stack for a state that has GOTO transitions on synchronization variables (like `Stmt` , `DeclPart`), skips tokens to their FOLLOW sets, pushes GOTO states, and resumes.

2.6 Symbol Table Manager (`symbol_table.cpp`)

- Organizes identifiers in a chain of **Scope** blocks.
- Each scope block has a hash map mapping strings to symbols.
- Scopes are linked: nested functions point to their outer scopes (parent pointers).
- Lookups are recursive (looking in current scope, then parent, up to global).
- Keeps a historical record of all scope states to output when compilation completes.

2.7 Error Handler (`error_handler.cpp`)

- Accumulates `Lexical` , `Syntactic` , and `Semantic` error structures.
- Prints a clear summary logging the line number, column number, and error details.

3. SLR(1) LR Table Construction Details

3.1 LR(0) Item Closures and Goto Transitions

An LR(0) item is a production rule containing a dot (`.`) representing the progress of parsing.

- **Closure(I)**: For every item $A \rightarrow \alpha \cdot B \beta$ in I , we add all productions $B \rightarrow \gamma$ to the set I .
- **GOTO(I, X)**: Computes the closure of all items $A \rightarrow \alpha X \cdot \beta$ for all items in I containing $A \rightarrow \alpha \cdot X \beta$.

Starting with $I_0 = \text{Closure}(S' \rightarrow \cdot \text{Program})$, the engine traverses transitions for all terminal and non-terminal symbols to generate the complete DFA of states.

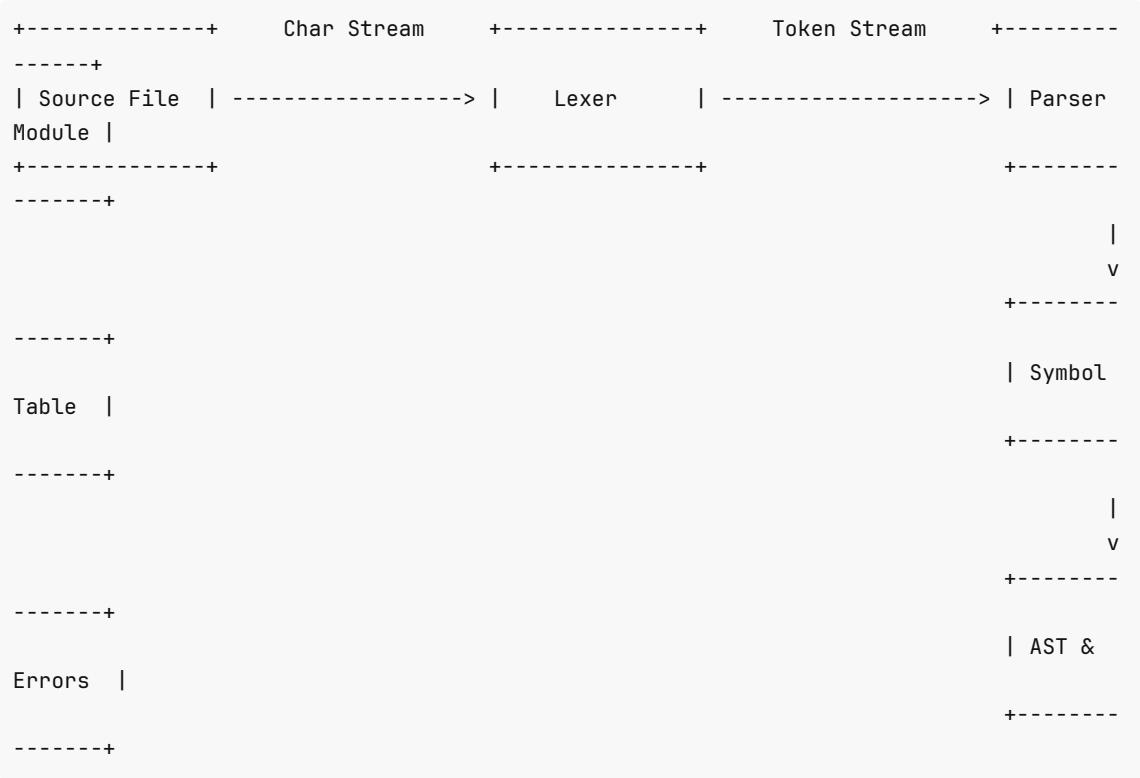
3.2 Action and Goto Action Assignments

For each state i in the DFA:

1. If item $A \rightarrow \alpha \cdot a \beta$ is in state i and $\text{GOTO}(i, a) = \text{state } j$ (where a is a terminal), we set $\text{Action}[i, a] = \text{Shift } j$.
2. If item $A \rightarrow \alpha \cdot$ is in state i (where $A \neq S'$), we set $\text{Action}[i, a] = \text{Reduce } A \rightarrow \alpha$ for all terminals $a \in \text{FOLLOW}[A]$.
3. If item $S' \rightarrow \text{Program} \cdot$ is in state i , we set $\text{Action}[i, \$] = \text{Accept}$.
4. If $\text{GOTO}(i, A) = \text{state } j$ (where A is a non-terminal), we set $\text{Goto}[i, A] = j$.

4. Discussion of Integration

All modules are driven by the wrapper program in `src/main.cpp` :



The driver opens the input file once, generates the grammar parameters, and then triggers each parser sequentially. The lexer is reset between parser runs. Detailed compilation outputs (tokens, AST, symbol table history, parser traces, and error logs) are written directly to the `output/` folder.

5. Limitations & Future Work

- Single-Dimensional Arrays Only:** Currently, only 1D arrays are supported. Support for multi-dimensional arrays (`array[1 .. 10, 1 .. 5]`) can be added by modifying the parser to match commas in indexing.
- Lack of Intermediate Code Generation:** The mini compiler performs frontend scanning, parsing, and semantic verification, but does not compile to three-address code (TAC) or assembly. Implementing target code generation is a key direction for future extensions.

6. References

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.
- Cooper, K. D., & Torczon, L. (2011). *Engineering a Compiler* (2nd ed.). Morgan Kaufmann.